MNIST Training Report for DQNN/IDQNN

Quantum Machine Learning

April 15, 2026

Inference with DQNN/IDQNN

Inference stage

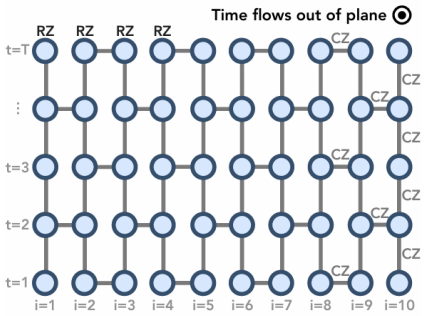
Given an input bitstring $x \in \{0,1\}^n$, the generative quantum model produces output bitstring $y \in \{0,1\}^n$ with parameters θ:

$$y \sim p_\theta(y \mid x).$$

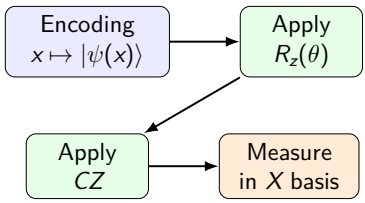
- Models: DQNN (Deep Quantum Neural Network) and shallow IDQNN (Instantaneous Deep Quantum Neural Network).
- They generate the same conditional distribution:

$$P_{\mathrm{IDQNN}}(y \mid x, \theta) = P_{\mathrm{DQNN}}(y \mid x, \theta).$$

IDQNN Definition and Architecture



2D shallow QNN architecture used by IDQNN.



$$|\psi_{\text{out}}\rangle = \left(\prod CZ\right)\left(\prod R_z(\theta)\right)|\psi(x)\rangle\,, \quad y \sim p_\theta(y \mid x)$$

Input-state encoding rule

Qubit i is initialized from $\{|0\rangle, |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\}$ according to bit x_i: $0 \mapsto |+\rangle$, $1 \mapsto |0\rangle$.

Task 1 Learning Algorithm: Dataset Construction

1. Sample N_{sample} input bitstrings from $D(x)$.
2. Fix ideal parameters θ_{real}.
3. Generate the dataset under ideal parameters:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N_{\text{sample}}}.$$

Training objective

Estimate θ from $\mathcal{D}$ such that θ approaches θ_{real}.

Task 1 Learning Algorithm: Block-wise Estimation

Dataset generation

Using ideal parameters θ^* and IDQNN circuit, generate dataset:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N_{\text{sample}}}.$$

Learning stage: estimate each parameter block-wise.

- For each parameter at position (i, j), filter samples satisfying:

  $$x_{i,j} = 0, \qquad x_{\mathcal{N}(i,j)} = 1,$$

  where $\mathcal{N}(i, j)$ denotes positions connected via CZ gates.
- Compute the parameter estimate from the filtered samples.

$$\hat{\theta}_{i,j} = \arccos\left(1 - \frac{2}{N_{\text{sp}}} \sum_{t=1}^{N_{\text{sp}}} y_{(t)}^{(i,j)}\right).$$

Quantum advantage

New Question: MNIST Training Performance

Question

How well do DQNN/IDQNN train on a practical handwritten-digit task?

- MNIST training set: 60,000 samples; test set: 10,000 samples.
- Original image size: $28 \times 28 = 784$ grayscale pixels, normalized to $[0, 1]$.
- We construct a quantum-learning dataset $\mathcal{D}_{\mathrm{MNIST}}$ of bitstring pairs (x_i, y_i).

Key practical point

DQNN and IDQNN represent the same distribution, but runtime differs significantly in practice.

Runtime: DQNN vs IDQNN

- DQNN uses only m qubits; IDQNN uses $n_1 \times m$ qubits in the shallow representation.
- Measured sampling runtime in our implementation:

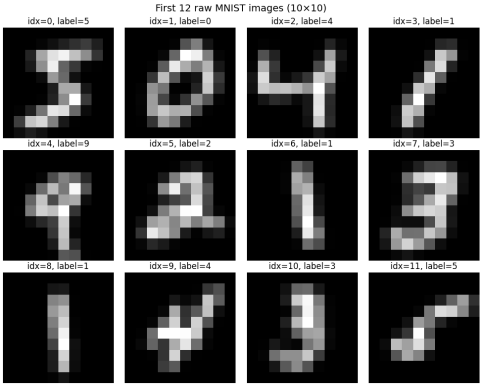
oprule Model & input size & Sampling time (single run)
DQNN, 10×10 ($n = 100$) & & 1.8 s
IDQNN, 5×5 ($n = 25$) & & 44.019 s

Implication

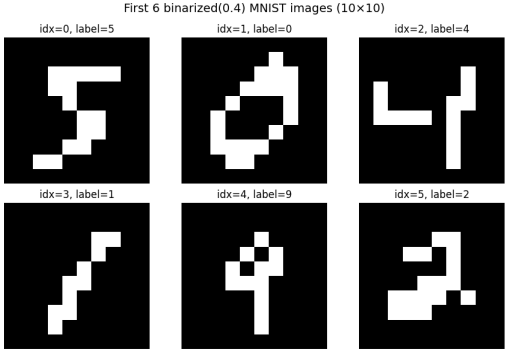
For MNIST-scale experiments, DQNN is much more computationally feasible for repeated sampling.

Preprocessing: Resize to 10×10 for Input x

- We reshape each MNIST image to a 10×10 grayscale image.
- The resulting 100-pixel vector is mapped to the input bitstring x.



Binarization and Threshold Choice



Example binarized samples.

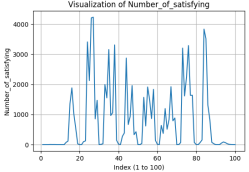
- Two thresholds were tested:

$$\tau \in \{0.5,\ 0.4\}.$$

- Pixel rule:

$$x_{\text{bin}} = \mathbb{I}(x > \tau).$$

- Different τ values change effective sparsity and filter-sample availability in training.

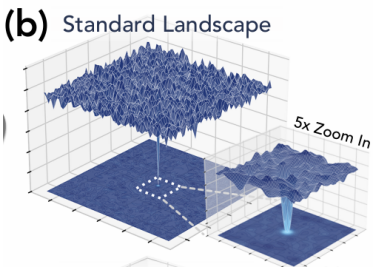


Label-Bitstring Encoding for y (0–9)

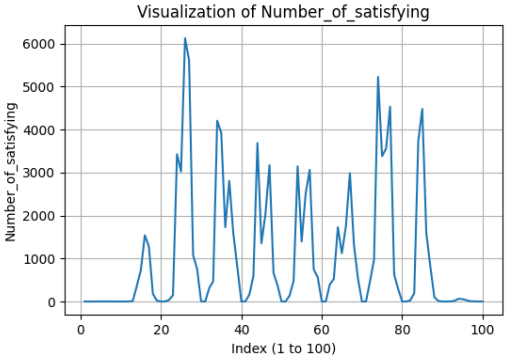
We encode each digit label using a 100-bit (10×10) diagonal code:

$$y_{i,j}^{(k)} = \begin{cases} 1, & j = (i+k) \bmod 10, \\ 0, & \text{otherwise.} \end{cases}$$

- Digit 0: main diagonal.
- Digit 1: first super-diagonal (cyclic).
- Digit 2: second super-diagonal (cyclic), etc.



Filter-Sample Statistics for Parameter Estimation



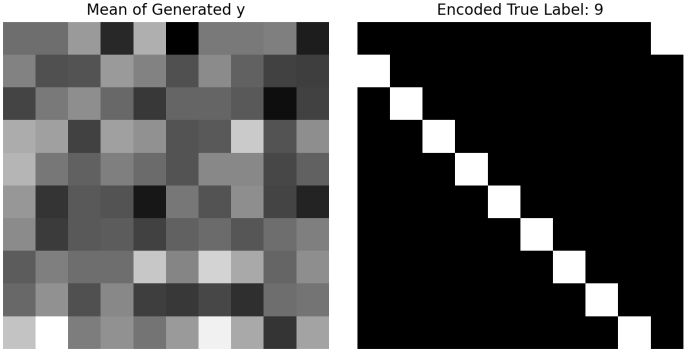
- We first count valid filter samples for each parameter position.
- Parameters with no valid samples are set to 0.

Observed in one run

Skipped bits ($N_{\mathrm{sp}} = 0$): 21
Indices (0-based):
[0,1,2,3,7,8,9,10,11,19,20,29,50,60,69,80,89,90,91,98

Generation on Test Set with Learned Parameters

- After estimating θ, we generate y on test inputs.
- Example: for test index 99, we sampled 1000 generated outputs and averaged the generated bits.
- The averaged generated pattern is compared with the true encoded label.



Conclusions

- We extended the inference+learning pipeline from synthetic data to MNIST-derived bitstring data.
- DQNN is significantly faster than IDQNN in our sampling tests, while preserving equivalent target distribution.
- Binarization threshold and filter-sample coverage strongly affect parameter estimation quality.
- Preliminary test-set generation shows meaningful alignment with encoded target labels.

Next steps

Improve sparse-filter handling, compare threshold strategies systematically, and benchmark against classical baselines on the same encoded task.